

Arrays (1D, 2D)

An array is a fixed-size container that holds multiple values of the same data type under a single variable name.

When to use:

Use arrays when you need to store a collection of related values of same type - instead of creating separate variables for each (num1, num2, ...).

Use 2D arrays when data naturally forms a grid/table (rows and columns) - like a seating chart, matrix, or board game.

How to use:

1D Array

```
int[] marks = {90, 85, 78, 92, 60};  
S.O.P(marks[0]);  
marks[2] = 80;  
int length = marks.length;  
for (int i = 0; i < marks.length; i++) {  
    S.O.P(marks[i]);  
}
```

2D Array

```
int[][] grid = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};  
S.O.P(grid[1][2]);  
for (int i = 0; i < grid.length; i++)  
    for (int j = 0; j < grid[i].length; j++)  
        S.O.P(grid[i][j] + " ");  
S.O.P();
```


Alternative declarations:
`int[] arr = new int[5];`

Where to use:

- * 1D arrays: lists of scores, prices, names, temperatures - anything linear
- * 2D arrays: matrices, tic-tac-toe boards, seat maps, Spreadsheet-like data.
- * Almost always paired with loops (for) since you rarely access array elements one by one manually

Why does indexing start at 0, not 1?

- * indexing starts at 0-index = memory offset from start

- * Array size is fixed - enables instant (O(1)) access via direct memory calculation:

resizable version = ArrayList.

- * `length` is a field, not a method - size stored as metadata, no computation needed.

- * 2D array = "array of arrays" - each stored as row is its own `int[]`, so rows can have different lengths (jagged arrays).

Real-life Example

Scenario

Marks of 30 students

Chessboard / tic-tac-toe

Theatre seating (rows x column
seats)

Array type

1D

2D

2D